Mini-project workshop

M.I. Capel

ETS Ingenierías Informática y
Telecomunicación
Departamento de Lenguajes y Sistemas Informáticos
Universidad de Granada
Email: manuelcapel@ugr.es
http://lsi2.ugr.es/mcapel/

24 de marzo de 2026
Máster Universitario en Desarrollo de Software

What You Must Implement

Core components:

1. Token encoding and decoding
2. Embedding layers (token + positional)
3. Self-attention head (Q, K, V, masking)
4. Multi-head attention
5. Transformer block (attention + feed-forward)
6. Training loop (forward + backward + optimizer)
7. Text generation (autoregressive sampling)

Expected outcome:

- A working character-level Transformer
- Ability to generate coherent text

Pipeline

The key chain to implement:

- encode(text)
- tensor of token IDs
- nn.Embedding
- sequence of vectors
- self-attention / Transformer blocks
- linear layer
- logits over vocabulary
- predicted next token

Extracting the vocabulary

In Python the following functions are convenient to do the vocabulary extraction job:

- `set(text)` → gets unique characters from your text (removes duplicates)
- `list(...)` → converts it back to a list
- `sorted(...)` → sorts characters (important for reproducibility!)
- `vocab_size` = number of unique characters

Example:

```
text = "hello"
set(text) = {'h', 'e', 'l', 'o'}
chars = sorted(list(set(text)))
chars = ['e', 'h', 'l', 'o']
vocab_size = len(chars)

```

Encoding/Decoding numbers

- One dictionary `stoi` maps each character to a number

```
        stoi = {ch: i for i, ch in enumerate(chars)}
        stoi = {'e': 0, 'h': 1, 'l': 2, 'o': 3}

```

[style=Python, basicstyle=]

- `second one itos is the reverse mapping`

```
        itos = {i: ch for i, ch in enumerate(chars)}
        itos = {0: 'e', 1: 'h', 2: 'l', 3: 'o'}

```

Why both `stoi` and `itos` ?

- Model works with numbers
- Humans read text
- So we need both directions

Encoding/Decoding numbers

- Encoding text → numbers

```
def encode(s: str):
    return [stoi[c] for c in s]
encode("hello")
#stoi = {'e': 0, 'h': 1, 'l': 2, 'o': 3}
#returns: [1, 0, 2, 2, 3]

```

- Decoding numbers → text

```
def decode(indices):
    return "".join(itos[i] for i in
indices)
decode([1, 0, 2, 2, 3])
#returns: "hello"

```

Converting everything to a tensor

- We build a dictionary of characters, turn text into numbers, and feed it to the model
- The whole initial block must be: Text $\rightarrow$ Numbers $\rightarrow$ Tensor:
  - Neural networks cannot read characters
  - They only understand numbers
- Wrap it into as PyTorch tensor:

```
1                data = torch.tensor(encode(text), dtype=
    torch.long)
2
```

- Why `dtype=torch.long` is necessary? Because
  - PyTorch embedding layers expect integer indices
  - `long = integer` type used for indexing

Step	Input	Output
encode	"hello"	[1, 0, 2, 2, 3]
tensor	[1, 0, 2, 2, 3]	tensor([...])
decode	[1, 0, 2, 2, 3]	"hello"

The role of `nn.Embedding`

Key idea

`encode` turns characters into integer IDs, and **`nn.Embedding`** turns those IDs into trainable vectors that the Transformer can actually understand.

- Pass indices into the embedding: `x = embedding(data)`
- `x` is no longer a list of integers, but it is a Python tensor of shape: `(sequence_length, embedding_dim)`

Example:

```
    data = tensor([1, 0, 2, 2, 3])
    x = embedding(data)
    tensor([
            [ 0.9,  0.3, -0.2,  0.4],   # 'h'
            [ 0.2, -0.5,  0.1,  0.7],   # 'e'
            [-0.1,  0.8,  0.6, -0.3],   # 'l'
            [-0.1,  0.8,  0.6, -0.3],   # 'l'
            [ 0.5, -0.7,  0.2,  0.1]    # 'o'
])
```

The role of `nn.Embedding`–II

`nn.Embedding` is learnable

- At the start, the vectors in the tensor are random
- During training, PyTorch updates them to get useful representations in similar contexts
- Model learns the embedding table with the rest of the Transformer

Why not use the raw integers directly?

- Because integers like: `[1, 0, 2, 2, 3]` do not contain meaningful geometric information
- The embedding layer learns a useful vector representation

Why repeated tokens give repeated vectors

Identical characters in the sequence map to the same index number and then to the same embedding vector.
Later, positional encoding helps to distinguish them, or otherwise the model would not know where each token occurs.

```python
import torch
import torch.nn as nn
text = "hello"
chars = sorted(list(set(text)))
stoi = {ch: i for i, ch in enumerate(chars)}
def encode(s):
    return [stoi[c] for c in s]
data = torch.tensor(encode(text), dtype=torch.long)
embedding = nn.Embedding(len(chars), 4)
x = embedding(data)
print("Token IDs:", data)
print("Embedded shape:", x.shape)
print(x)
```

```
Token IDs: tensor([1, 0, 2, 2, 3])
Embedded shape: torch.Size([5, 4])
tensor([[-1.2294,  0.5638, -2.0601,  1.4873],
        [-0.1288, -0.7823, -0.5898, -0.6979],
        [ 1.5934,  0.2459, -0.0068,  0.1177],
        [ 1.5934,  0.2459, -0.0068,  0.1177],
        [-0.2643, -0.7339, -2.1436,  0.1371]], grad_fn=<
    EmbeddingBackward0>)
```

From Tokens to a Sequence of Vectors

After tokenisation and embedding:

- Input text is converted into token indices:

  "hello" $\rightarrow [1, 0, 2, 2, 3]$

- Each token is mapped to a vector:

  Embedding $\rightarrow x_{\text{token}} \in \mathbb{R}^{T \times d_{\text{model}}}$

- **Positional information is added:**

  $$x = x_{\text{token}} + x_{\text{pos}}$$

  where x_{pos} encodes the position of each token in the sequence.

- Final representation:

$$x = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \\ \vdots \\ \mathbf{x}_T \end{bmatrix} \in \mathbb{R}^{T \times d_{\text{model}}}$$

From Tokens to a Sequence of Vectors-II

Key idea:

- Each vector $\mathbf{x}_t$ encodes:
  - token identity (embedding)
  - token position (positional encoding)
- This allows the model to reason about **order in sequences**.

Next step: Project into attention spaces

$$Q = xW_Q \qquad K = xW_K \qquad V = xW_V$$

Model Configuration and Hyperparameters

Training and model parameters defined in a configuration object:

```python
@dataclass
class Config:
    batch_size: int = 32
    block_size: int = 64
    learning_rate: float = 3e-4
    n_embd: int = 128
    n_head: int = 4
    n_layer: int = 3
    dropout: float = 0.1
    train_split: float = 0.9
    seed: int = 42
    generate_tokens = 300
```

Instead of learning from one sequence at a time, the model learns from 32 different text fragments simultaneously:

- Centralised definition of all experiment parameters
- Easy to modify and reproduce experiments
- Controls both:
  - model architecture ($n_{\text{embd}}, n_{\text{head}}, n_{\text{layer}}$)
  - training process (batch_size, learning_rate)

Reproducibility:

```python
torch.manual_seed(cfg.seed)
```

Dropout: Preventing Overfitting

Definition:

- Dropout randomly sets a fraction of activations to zero during training

Example:

$$[0.2, 0.8, 0.5, 0.1] \rightarrow [0.2, 0.0, 0.5, 0.0]$$

Why use dropout?

- Prevents overfitting
- Forces the model to learn robust representations

Parameter:

$$\text{dropout} = 0.1 \Rightarrow 10\% \text{ of neurons are dropped}$$

Important:

- Applied only during training
- Disabled during inference

Batch Size: Multiple Sequences in Parallel

Definition:

- `batch_size` = number of sequences processed simultaneously
- Each sequence has length `block_size`

Example: batch_size $= 3$, block_size $= 4$

$$x = \begin{bmatrix} h & e & l & l \\ o & \_ & w & o \\ r & l & d & . \end{bmatrix} \quad y = \begin{bmatrix} e & l & l & o \\ \_ & w & o & r \\ l & d & . & \_ \end{bmatrix}$$

Interpretation:

- Each row = one training sequence
- Each column = one prediction step
- The model predicts the next token at every position

In practice:

$x, y \in \mathbb{R}^{\text{batch\_size} \times \text{block\_size}} \quad \Rightarrow \quad (32, 64)$

- Many sequences are processed in parallel $\rightarrow$ faster and more stable training

This enables autoregressive learning

Splitting the Dataset

We divide the dataset into training and validation sets:

```
n = int(cfg.train_split * len(data))
train_data = data[:n]
val_data = data[n:]
```

Example:

$$\text{data} = [x_1, x_2, \ldots, x_N]$$

$$\text{train} = [x_1, \ldots, x_{0.9N}] \quad \text{val} = [x_{0.9N+1}, \ldots, x_N]$$

Why do we split?

- Training set $\rightarrow$ used to learn model parameters
- Validation set $\rightarrow$ used to evaluate generalisation

Sanity check:

```
print(len(train_data), len(val_data))
```

Batch Generation: Conceptual View

Goal: Create training examples for next-token prediction

Given a sequence of tokens:

$$[x_1, x_2, x_3, x_4, x_5, \ldots]$$

We extract input-output pairs:

$$x = [x_1, x_2, x_3, x_4] \quad y = [x_2, x_3, x_4, x_5]$$

Key idea:

- Input = current tokens
- Target = next token at each position

With batching:

- We generate many such sequences in parallel
- Each batch contains multiple random chunks of text

Result:

$$x, y \in \mathbb{R}^{\text{batch\_size} \times \text{block\_size}}$$

Batch Generation for Autoregressive Training

Goal: Create input-output pairs for next-token prediction

```python
def get_batch(split):
    source = train_data if split == "train" else val_data
    ix = torch.randint(len(source) - block_size - 1,
                       (batch_size,))

    x = torch.stack([source[i:i + block_size] for i in ix])
    y = torch.stack([source[i + 1:i + block_size + 1]
                     for i in ix])

    return x, y
```

Key idea: shifting the sequence

$$x = [x_1, x_2, x_3, x_4] \quad y = [x_2, x_3, x_4, x_5]$$

- Input: current tokens
- Target: next token at each position

How `get_batch()` Works (Concrete Example)

Given:

$$\text{data} = [10, 11, 12, 13, 14, 15, 16, 17, 18, 19]$$

$$\text{batch\_size} = 2, \quad \text{block\_size} = 4$$

Step 1: Random starting indices

$$\texttt{ix} = [2,\ 5]$$

Step 2: Build input sequences

$$x = \begin{bmatrix} 12 & 13 & 14 & 15 \\ 15 & 16 & 17 & 18 \end{bmatrix}$$

Step 3: Build target sequences (shifted by +1)

$$y = \begin{bmatrix} 13 & 14 & 15 & 16 \\ 16 & 17 & 18 & 19 \end{bmatrix}$$

Key idea:

- Each row is one training sequence
- The model learns: *predict the next token*

$$[12, 13, 14, 15] \rightarrow [13, 14, 15, 16]$$

One Masked Self-Attention Head

Input: $x \in \mathbb{R}^{B \times T \times C}$

- B: batch size
- T: sequence length
- C: embedding dimension

Linear projections: $Q = xW_Q, \quad K = xW_K, \quad V = xW_V$

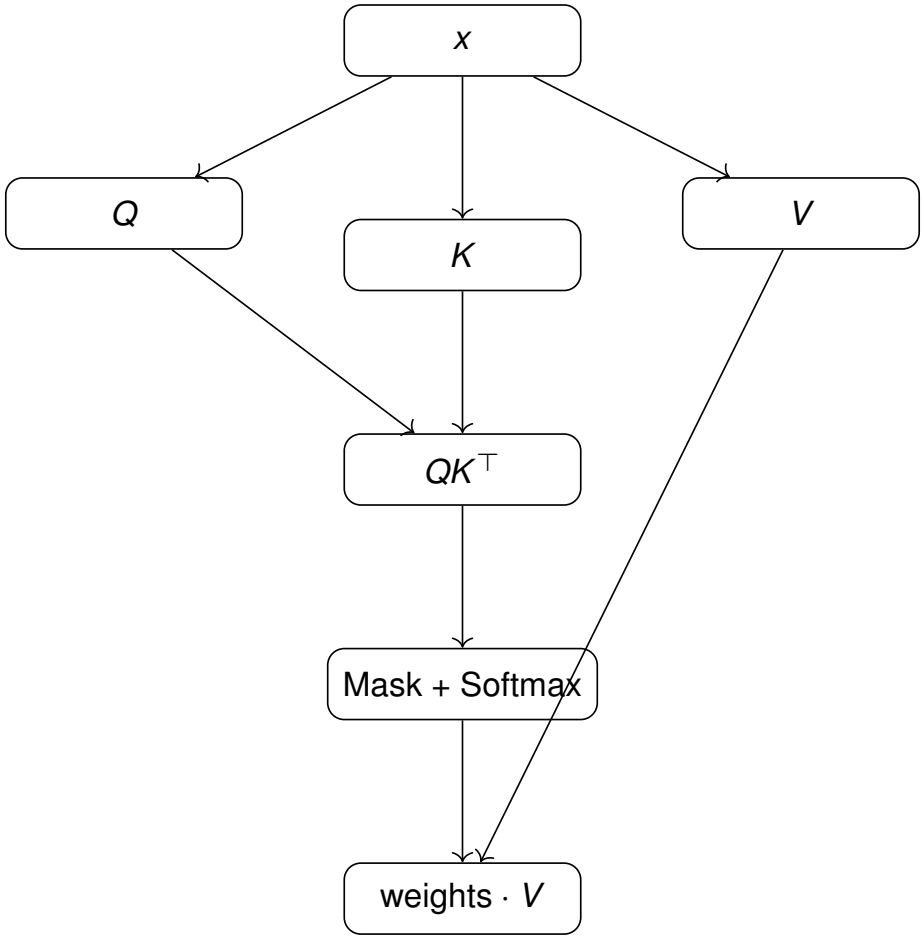
- Queries (Q): what each token is looking for
- Keys (K): what each token offers
- Values (V): information to be aggregated

Causal mask:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

- Prevents attending to future tokens
- Ensures autoregressive behaviour

Inside One Attention Head (Diagram)



Key idea:

- Queries interact with keys to compute attention weights
- Values are combined using these weights
- Mask ensures no access to future tokens

One Masked Self-Attention Head

```python
class Head(nn.Module):
    def __init__(self, head_size: int):
        super().__init__()
        self.key = nn.Linear(cfg.n_embd, head_size, bias=False)
        self.query = nn.Linear(cfg.n_embd, head_size, bias=False)
        self.value = nn.Linear(cfg.n_embd, head_size, bias=False)
        self.dropout = nn.Dropout(cfg.dropout)
        self.register_buffer(
            "tril",
            torch.tril(torch.ones(cfg.block_size, cfg.block_size))
        )
    def forward(self, x, return_weights=False):
       ...
        return out
```

What One Attention Head Does

Input to the head:

$$x \in \mathbb{R}^{B \times T \times n\_embd}$$

where:

- B = batch size
- T = sequence length
- *n_embd* = embedding dimension of each token

Step 1: project each token embedding into three new spaces

$$K = xW_K, \qquad Q = xW_Q, \qquad V = xW_V$$

with $K, Q, V \in \mathbb{R}^{B \times T \times \texttt{head\_size}}$

Step 2: compare queries and keys

$$\text{weights} = \frac{QK^\top}{\sqrt{\texttt{head\_size}}}$$

This gives weights $\in \mathbb{R}^{B \times T \times T}$ Each token gets one score for every token in the sequence (**forward pass**).

`n_embd` vs. d_k in Attention

Two related dimensions, but not always the same:

$$\boxed{\text{token embedding}} \in \mathbb{R}^{n\_embd} \quad \Longrightarrow \quad \boxed{Q} \in \mathbb{R}^{d_k}, \quad \boxed{K} \in \mathbb{R}^{d_k}$$

- `n_embd`: size of the full token representation
- d_k: size of the query/key vectors used in attention

Attention uses:

$$\text{scores} = \frac{QK^\top}{\sqrt{d_k}}$$

Typical cases:

$$\boxed{\text{Single-head: } d_k = n\_embd} \quad \boxed{\text{Multi-head: } d_k = \frac{n\_embd}{n\_head}} (= headsize)$$

Example: If `n_embd = 64` and `n_head = 4`, then each head uses

$$d_k = 16$$

What One Attention Head Does-II

(forward pass) continues . . .
Step 3: apply causal mask Future positions are blocked so each token can only attend to the past.

Step 4: softmax Convert scores into attention probabilities.

Step 5: weighted sum of values

$$\text{out} = \text{weights} \cdot V$$

so

$$\text{out} \in \mathbb{R}^{B \times T \times \texttt{head\_size}}$$

Attention Computation (Forward Pass) summary

Step 1: Similarity scores

$$\text{scores} = QK^\top$$

Step 2: Scaling

$$\text{scores} = \frac{QK^\top}{\sqrt{d_k}}$$

Step 3: Apply causal mask

- Future positions $\rightarrow -\infty$

Step 4: Softmax: weights $=$ softmax(scores)

- Each row sums to 1
- Represents attention distribution

Step 5: Weighted sum output $=$ weights $\cdot\, V$

Interpretation:

- Each token attends to previous tokens
- Aggregates relevant information

Why Multi-Head Attention?

A single attention head learns one attention pattern

Multi-head attention:

- runs several attention heads in parallel
- allows the model to capture different relationships at the same time
- combines their outputs into a richer representation

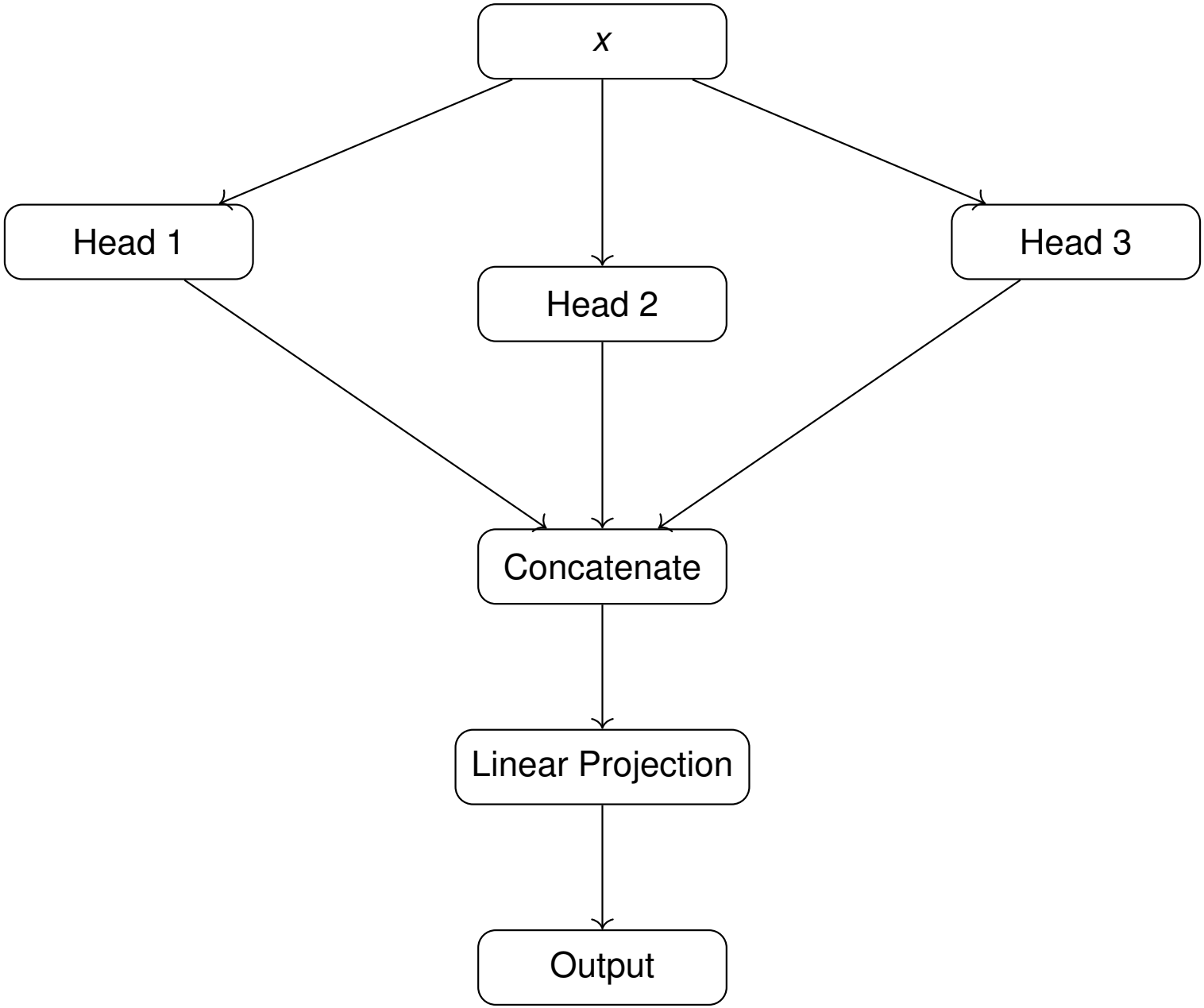
Examples of different heads:

- local context
- long-range dependencies
- punctuation or separators
- repeated patterns

Key idea:

Multi-head attention $=$ several self-attention heads in parallel

Multi-Head Attention: Diagram



Multi-Head Attention: Big Picture

Input:

$$x \in \mathbb{R}^{B \times T \times n\_embd}$$

Each head receives the same input and produces:

$$(B, T, \texttt{head\_size})$$

Then:

concatenate all head outputs $\rightarrow (B, T, n\_head \cdot \texttt{head\_size})$

Finally, apply a linear projection back to the model dimension:

$$(B, T, n\_embd)$$

Interpretation:

- each head gives one “view” of the sequence
- concatenation collects all views
- projection mixes them into one representation

Structure of `MultiHeadAttention`

```
class MultiHeadAttention(nn.Module):
    def __init__(self, num_heads: int, head_size: int):
        super().__init__()
        self.heads = nn.ModuleList(
            [Head(head_size) for _ in range(num_heads)]
        )
        self.proj = nn.Linear(num_heads * head_size, cfg.n_embd)
        self.dropout = nn.Dropout(cfg.dropout)
```

Components:

- `self.heads`: a list of independent attention heads
- each head receives the same input x
- each head returns output of shape `(B,T,head_size)`
- concatenation gives shape `(B,T,num_heads * head_size)`
- `self.proj` maps the concatenated result back to `cfg.n_embd`
- `self.dropout`: regularization after projection

Why `ModuleList`?

- registers all heads as trainable submodules in PyTorch

Forward Pass of `MultiHeadAttention`

```
def forward(self, x, return_weights=False):
        if return_weights:
            ...
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        out = self.dropout(self.proj(out))
        return out
```

Step by step:

1. `h(x)`: each head processes the same input x
2. each head produces an output of shape $(B, T, \text{head\_size})$
3. `torch.cat(..., dim=-1)` concatenates along the dimension
4. result: $(B, T, n\_head \cdot \texttt{head\_size})$
5. a linear projection maps the result back to n_{embd}
6. `self.proj(out)` mixes the information from all heads
7. final output shape: (B, T, n_embd)

Shape example:

$$n_{\text{embd}} = 128, \quad n_{\text{head}} = 4, \quad \text{head\_size} = 32$$

Shape Flow in Multi-Head Attention

$$x : (B, T, n\_embd)$$

$$\underbrace{\text{Head } 1(x)}_{(B,T,\texttt{head\_size})} \quad , \quad \underbrace{\text{Head } 2(x)}_{(B,T,\texttt{head\_size})} \quad , \quad \ldots$$

After concatenation:

$$(B, T, n\_head \cdot \texttt{head\_size})$$

After projection:

$$(B, T, n\_embd)$$

Typical setup:

$$\texttt{head\_size} = \frac{n\_embd}{n\_head}$$

so the concatenated size matches the model size again.

The `return_weights=True` Branch

```python
if return_weights:
    outs = []
    weights = []
    for h in self.heads:
        out_h, wei_h = h(x, return_weights=True)
        outs.append(out_h)
        weights.append(wei_h)
    out = torch.cat(outs, dim=-1)
    out = self.dropout(self.proj(out))
    return out, torch.stack(weights, dim=0)
```

- each head returns:
  - `out_h`: head output
  - `wei_h`: attention matrix
- `weights` stores the attention matrix of each head
- `torch.stack(weights, dim=0)` groups them together

Useful for:

- visualising attention
- interpreting what each head is focusing on

What Shape Do the Returned Weights Have?

For one head:

$$\texttt{wei\_h} \in \mathbb{R}^{B \times T \times T}$$

Interpretation:

- one attention matrix per sequence in the batch
- each row corresponds to one token
- each column corresponds to the token being attended to

After stacking all heads:

$$\texttt{weights} \in \mathbb{R}^{n\_head \times B \times T \times T}$$

This allows us to inspect how different heads attend differently.

Transformer Block: Interpretation

A Transformer block alternates between:

- **communication**: tokens exchange information through attention
- **computation**: each token refines its representation through a neural network

Compact view:

$$x \to \text{LayerNorm} \to \text{MultiHeadAttention} \to +x \to$$
$$\text{LayerNorm} \to \text{FeedForward} \to +x$$

Output shape remains unchanged:(B, T, n_{embd})

This allows multiple blocks to be stacked one after another.

Complete **Pipeline:**

$$\text{input} \to \text{LayerNorm} \to \text{Multi-Head Attention} \to \text{Residual Add}$$
$$\to \text{LayerNorm} \to \text{FeedForward} \to \text{Residual Add}$$

Structure of One Transformer Block

```
class Block(nn.Module):
    def __init__(self, n_embd: int, n_head: int):
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedForward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)
```

Components:

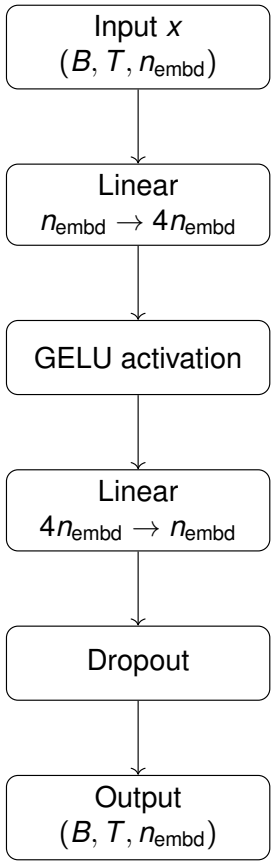
- multi-head self-attention
- feed-forward network
- two layer normalizations

Head size:

$$\text{head\_size} = \frac{n_{\text{embd}}}{n_{\text{head}}}$$

Feed-Forward Network: `self.net(x)`

Idea: Each token is processed by a small neural network



Key idea:

- Applied **independently to each token**
- Adds nonlinear transformation after attention

The Feed-Forward Network

After attention, each token has gathered information from other tokens. Now we process each token representation with a small neural network:

$$\texttt{Linear}(\textit{n\_embd}, 4{\cdot}\textit{n\_embd}) \rightarrow \texttt{GELU} \rightarrow \texttt{Linear}(4{\cdot}\textit{n\_embd}, \textit{n\_embd})$$

Important:

- attention mixes information *across tokens*
- feedforward transforms features *within each token*

So:

- self-attention = communication
- feedforward = computation

Typical dimensional flow:

$$n_{\text{embd}} \rightarrow 4n_{\text{embd}} \rightarrow n_{\text{embd}}$$

$$128 \rightarrow 512 \rightarrow 128$$

Why Does FeedForward Use $4 \cdot n_{\text{embd}}$ Internally?

Input to FeedForward:

$$x \in \mathbb{R}^{B \times T \times n_{\text{embd}}}$$

The FeedForward block is:

$$\text{Linear}(n_{\text{embd}},\, 4n_{\text{embd}}) \rightarrow \text{GELU} \rightarrow \text{Linear}(4n_{\text{embd}},\, n_{\text{embd}})$$

So the feature dimension changes only *inside* the block:

$$(B, T, n_{\text{embd}}) \rightarrow (B, T, 4n_{\text{embd}}) \rightarrow (B, T, 4n_{\text{embd}}) \rightarrow (B, T, n_{\text{embd}})$$

Key idea:

- the first linear layer *expands* the representation
- GELU applies a non-linear transformation
- the second linear layer *compresses* it back

Example: if $n_{\text{embd}} = 64$

$$(B, T, 64) \rightarrow (B, T, 256) \rightarrow (B, T, 64)$$

Conclusion: the FeedForward network does *not* permanently change the model dimension; it only uses a larger hidden space internally to learn richer transformations.

FeedForward Line by Line

```
class FeedForward(nn.Module):
    def __init__(self, n_embd: int):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.GELU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(cfg.dropout),
        )

    def forward(self, x):
        return self.net(x)
```

- first linear layer expands the feature dimension
- `GELU()` introduces non-linearity
- second linear layer returns to the original size
- dropout regularises the network

Output shape remains: (B, T, n_embd)

The Full Transformer Block Line by Line

```
class Block(nn.Module):
"Transformer block:communication followed by computation"
    def __init__(self, n_embd: int, n_head: int):
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedForward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)
    def forward(self, x):
       ...
       return x
```

- `head_size = n_embd // n_head` the embedding dimension is split across heads
- `self.sa`: communication between tokens
- `self.ffwd`: computation on each token
- `ln1`, `ln2`: normalisation for stable training

Forward Pass of the Full Transformer Block

```
def forward(self, x):
    x = x + self.sa(self.ln1(x))
    x = x + self.ffwd(self.ln2(x))
    return x
```

- first: normalise and apply self-attention
- then: add the original input (residual connection)
- second: normalise and apply feedforward network
- again: add the current representation (residual connection)

Interpretation:

- the block first lets tokens communicate
- then it refines each token locally

Attention vs Feed-Forward

Self-attention: communication across tokens

- each token can look at other tokens
- captures relationships inside the sequence

Feed-forward: computation within each token

- each token is transformed independently
- adds expressive nonlinear processing

Key idea:

Attention $=$ tokens communicate

FeedForward $=$ each token computes

Why Residual Connections and LayerNorm?

Residual connections:

$$x + f(x)$$

help preserve the original representation and improve gradient flow.

Layer normalization:

- stabilises activations
- improves optimisation
- makes training deeper models easier

Practical intuition:

- residuals = “refine the representation, do not replace it completely”
- layer norm = “keep values well-scaled during training”

Concrete Numerical Example

Suppose: $B = 32, \quad T = 8, \quad n\_embd = 64, \quad n\_head = 4$

Then:

$$\texttt{head\_size} = \frac{64}{4} = 16$$

Input to the block: $x \in \mathbb{R}^{32\times 8\times 64}$

Each head outputs: $(32, 8, 16)$

Four heads together:

$$4 \times (32, 8, 16)$$

After concatenation: $(32, 8, 64)$

After projection: $(32, 8, 64)$

FeedForward:

$$(32, 8, 64) \to (32, 8, 256) \to (32, 8, 64)$$

Final block output: $(32, 8, 64)$

Summary: Core Transformer Components

- **Head**: one masked self-attention mechanism
- **MultiHeadAttention**: several heads in parallel, then combine
- **FeedForward**: nonlinear per-token transformation
- **Block**: attention + feed-forward + layer normalization + residuals

Takeaway:

A Transformer block lets tokens first communicate with each other and then update their own internal representation.

CharTransformer: Overview

Goal: Character-level language modelling

Task:

- Given a sequence of characters
- Predict the next character at each position

Example:

input: *h e l l* $\Rightarrow$ target: *e l l o*

Model components:

- Token embeddings (what)
- Positional embeddings (where)
- Transformer blocks (processing)
- Output layer (prediction)

From Transformer Block to Full Character-Level Language Model

We already know how one Transformer block works:

- multi-head self-attention lets tokens communicate
- feedforward transforms each token representation
- residual connections and layer normalization stabilise training

Now we build the full model by stacking several blocks.

Goal of the full model:

- input: a sequence of character IDs
- output: scores for the next character at each position

This is the role of the `CharTransformer` class.

Overall Architecture of `CharTransformer`

token IDs $\rightarrow$ token embeddings

$+$ position embeddings $\rightarrow$ stack of Transformer blocks $\rightarrow$

final LayerNorm $\rightarrow$ linear layer to vocabulary $\rightarrow$ logits

Interpretation:

- token embeddings encode *what* each character is
- position embeddings encode *where* it is in the sequence
- Transformer blocks refine the sequence representation
- the final linear layer predicts the next-character scores

Model Architecture

```
self.token_embedding_table = nn.Embedding(vocab_size, cfg.n_embd)
self.position_embedding_table= nn.Embedding(cfg.block_size, cfg.n_embd)

self.blocks = nn.Sequential(
    *[Block(cfg.n_embd, cfg.n_head) for _ in range(cfg.n_layer)])
self.ln_f = nn.LayerNorm(cfg.n_embd)
self.lm_head = nn.Linear(cfg.n_embd, vocab_size)
self.apply(self._init_weights)
```

- `token_embedding_table`: maps token IDs to vectors
- `position_embedding_table`: adds position information
- `blocks`: stack of Transformer blocks
- `ln_f`: final normalization
- `lm_head`: maps final features to vocabulary scores

Model Architecture-II

Pipeline:

tokens $\rightarrow$ embeddings $\rightarrow$ Transformer blocks $\rightarrow$ logits

```
    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)
```

Forward Pass of `CharTransformer`

```
def forward(self, idx, targets=None):
    B, T = idx.shape
    tok_emb = self.token_embedding_table(idx)
    pos = torch.arange(T, device=idx.device)
    pos_emb = self.position_embedding_table(pos)
    x = tok_emb + pos_emb
    x = self.blocks(x)
    x = self.ln_f(x)
    logits = self.lm_head(x)
```

- convert token IDs into token embeddings
- build positional embeddings for positions $0, 1, \ldots, T-1$
- add token and position information
- pass through all Transformer blocks
- normalize and project to vocabulary scores

Final output:

$$\texttt{logits} \in \mathbb{R}^{B \times T \times \texttt{vocab\_size}}$$

When Is `forward()` Actually Executed in PyTorch?

In PyTorch, we usually do **not** call `forward()` explicitly.

Instead, when we call a module like a function, PyTorch automatically executes its `forward()` method.

Examples from this mini-project:

- `h(x)` → `Head.forward(x)`
- `self.sa(x)` → `MultiHeadAttention.forward(x)`
- `block(x)` → `Block.forward(x)`
- `model(xb, yb)` → `CharTransformer.forward(xb, yb)`

Execution chain in our model:

`model(xb, yb)` → `CharTransformer.forward()` → `Block.f`

Key idea: Calling a PyTorch module object automatically runs its `forward()` method.

Input and Embedding Shapes

Input:

$$\texttt{idx} \in \mathbb{N}^{B \times T}$$

Each entry is an integer ID of a character.

Token embeddings:

$$\texttt{tok\_emb} \in \mathbb{R}^{B \times T \times n_{\text{embd}}}$$

Position embeddings:

$$\texttt{pos\_emb} \in \mathbb{R}^{T \times n_{\text{embd}}}$$

Combined representation:

$$x = \texttt{tok\_emb} + \texttt{pos\_emb} \in \mathbb{R}^{B \times T \times n_{\text{embd}}}$$

Key idea:

- token embeddings say what the character is
- position embeddings say where it is

Shapes Through the Model

Tracking tensor shapes:

$$(B, T) \to (B, T, n_{\text{embd}}) \to (B, T, n_{\text{embd}}) \to (B, T, \text{vocab\_size})$$

Step-by-step:

- Input IDs $\to (B, T)$
- Embeddings $\to (B, T, n_{\text{embd}})$
- Transformer blocks $\to$ same shape
- Output logits $\to (B, T, \text{vocab\_size})$

Key idea:

The sequence length stays constant — only the representation changes.

Transformer Blocks

Stack of blocks:

$$x \to \text{Block}_1 \to \text{Block}_2 \to \cdots \to \text{Block}_L$$

Each block performs:

- Self-attention (communication between tokens)
- Feed-forward (per-token computation)

Output shape remains:

$$(B, T, n_{\text{embd}})$$

Key idea:

- Tokens exchange information and refine representations

Transformer blocks-II

```
    def forward(self, idx, targets=None):
        B, T = idx.shape
        tok_emb = self.token_embedding_table(idx)  # (B,T,n_embd)
        pos = torch.arange(T, device=idx.device)
        pos_emb = self.position_embedding_table(pos)  # (T,n_embd)
        x = tok_emb + pos_emb
        x = self.blocks(x)
        x = self.ln_f(x)
        logits = self.lm_head(x)  # (B,T,vocab_size)
        loss = None
        if targets is not None:
            B2, T2, C = logits.shape
            logits_flat = logits.view(B2 * T2, C)
            targets_flat = targets.view(B2 * T2)
            loss = F.cross_entropy(logits_flat, targets_flat)
        return logits, loss
```

Why Does the Output Have Shape $(B, T, \texttt{vocab\_size})$?

For each token position, the model predicts a score for *every possible next character*.

So if:

- batch size = B
- sequence length = T
- vocabulary size = V

then the output logits have shape:

$$(B, T, V)$$

Interpretation:

- one distribution per token position
- one score per vocabulary symbol

If $V = 65$, then each token position produces 65 scores.

Output Layer (Language Model Head)

Final transformation:

$$\text{logits} = xW_{\text{lm}}$$

$$(B, T, n_{\text{embd}}) \rightarrow (B, T, \text{vocab\_size})$$

Interpretation:

- For each position, the model predicts:
  - probability of each possible next character

Output:

- logits = unnormalized scores

Computing the Loss

```
loss = None
if targets is not None:
    B2, T2, C = logits.shape
    logits_flat = logits.view(B2 * T2, C)
    targets_flat = targets.view(B2 * T2)
    loss = F.cross_entropy(logits_flat, targets_flat)
```

Why flatten?

$$\texttt{logits} : (B, T, C) \rightarrow (B \cdot T, C)$$

$$\texttt{targets} : (B, T) \rightarrow (B \cdot T)$$

Cross-entropy expects:

- predictions of shape (N, C)
- targets of shape (N)

So each token position in the batch becomes one training example.

Interpretation:

- Model predicts next token at every position
- All predictions are evaluated together

Where Do `generate()` and `get_attention_weights()` Fit?

Training pipeline:

`idx, targets` → `forward()` → `logits, loss`

Generation pipeline:

`seed` → `generate()` → `forward()` `repeatedly` → new text

Inspection pipeline:

`idx` → `get_attention_weights()` → attention matrices

Key idea:

- `forward()` is the core model computation
- `generate()` and `get_attention_weights()` are auxiliary methods built on top of it

What is `generate(...)` Used For?

```
@torch.no_grad()
def generate(self, idx, max_new_tokens, temperature=1.0):
    self.eval()
    for _ in range(max_new_tokens):
     idx_cond = idx[:, -cfg.block_size:]
     logits, _ = self(idx_cond)
     logits = logits[:, -1, :] / temperature
     probs = F.softmax(logits, dim=-1)
     idx_next = torch.multinomial(probs, num_samples=1)
     idx = torch.cat((idx, idx_next), dim=1)
    return idx
```

Purpose: generate new text from a seed sequence.

At each step:

- keep the most recent context
- run the model on that context
- keep only the prediction for the last position
- convert logits into probabilities
- sample the next token
- append it to the sequence

What's `get_attention_weights(...)` Used For?

```python
@torch.no_grad()
def get_attention_weights(self, idx, block_number=0):
    self.eval()
    B, T = idx.shape
    tok_emb = self.token_embedding_table(idx)
    pos = torch.arange(T, device=idx.device)
    pos_emb = self.position_embedding_table(pos)
    x = tok_emb + pos_emb
    for i in range(block_number):
        x = self.blocks[i](x)
    block = self.blocks[block_number]
    out, weights = block.sa(block.ln1(x), return_weights=True)
    return weights
```

Purpose: inspect the attention matrices learned by the model.
What it does:

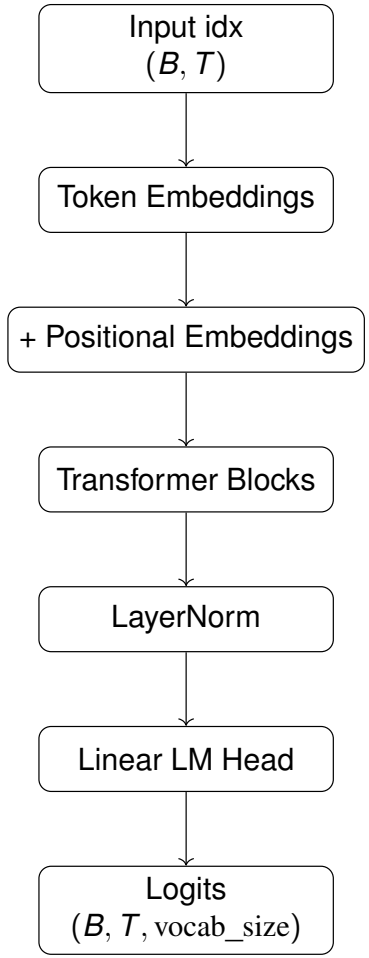
- rebuild the embedded input sequence
- pass it through the previous Transformer blocks
- stop at the chosen block
- return the attention weights of its self-attention module

Returned shape: (num_heads, B, T, T)

Interpretation: for visualisation, analysis, and understanding what each head is attending to

Forward Pass Summary

Full pipeline:



Output: $(B, T, \text{vocab\_size})$

Text Generation (Autoregressive)

Loop:

- take current sequence
- predict next-token distribution
- sample a token
- append it

Steps:

$$\text{logits} \rightarrow \text{softmax} \rightarrow \text{sampling}$$

Temperature:

- low $\rightarrow$ more deterministic
- high $\rightarrow$ more random

Key idea:

- generate one token at a time

Generation and Final Numerical Example

```
idx_cond = idx[:, -cfg.block_size:]
logits, _ = self(idx_cond)
logits = logits[:, -1, :] / temperature
probs = F.softmax(logits, dim=-1)
idx_next = torch.multinomial(probs, num_samples=1)
idx = torch.cat((idx, idx_next), dim=1)
```

Generation:

- keep the most recent context
- run the model
- keep only the last position
- convert logits into probabilities
- sample one new token
- append it to the sequence

Generation and Final Numerical Example-II

Concrete numerical example:
Suppose:

$$B = 32, \quad T = 8, \quad n_{\text{embd}} = 64, \quad n_{\text{head}} = 4, \quad V = 65$$

Then:

$$\texttt{idx} : (32, 8) \to \texttt{tok\_emb} : (32, 8, 64)$$

$$\texttt{pos\_emb} : (8, 64) \to x : (32, 8, 64) \to \text{blocks} : (32, 8, 64)$$

$$\texttt{logits} : (32, 8, 65)$$

For the loss:

$$(32, 8, 65) \to (256, 65), \qquad (32, 8) \to (256)$$

From Model Definition to Training Script

Up to now, we have studied the *internal structure* of the Transformer:

- token and positional embeddings
- self-attention and multi-head attention
- feedforward networks
- Transformer blocks

Now we switch to the execution workflow.

The training script shows how the model is actually used:

1. build the model
2. evaluate it periodically
3. train it with backpropagation
4. use it to generate text

Key idea: the class `CharTransformer` defines the model, while the script defines how to use it.

Step 1: Build the Model

```python
model = CharTransformer().to(device)
#number of elements stored in p: PyTorch function
num_params = sum(p.numel() for p in model.parameters())
print(model)
print()
print(f"Number of parameters: {num_params:,}")
```

- `CharTransformer()` creates the full model
- `.to(device)` moves it to CPU or GPU
- `model.parameters()` gives all trainable weights
- counting parameters helps us understand the size of the model

Interpretation: this is the setup phase before training starts.

Step 2: `estimate_loss()`

```python
@torch.no_grad() #avoids gradient computation
def estimate_loss():
    model.eval() #disables dropout
    out = {}
    for split in ["train", "val"]:
        losses = torch.zeros(cfg.eval_iters)
        for k in range(cfg.eval_iters):
            X, Y = get_batch(split)
            _, loss = model(X, Y)
            losses[k] = loss.item()
        out[split] = losses.mean().item()
    model.train()
    return out
```

`estimate_loss()` as an evaluation helper: Why Do We Need It?

Main idea:

- training loss from a single batch is noisy
- this helper averages loss over several batches
- it does so for both training and validation data

Purpose: obtain a more stable estimate of model quality during training.

Why Measure Both Train and Validation Loss?

Training loss:

- tells us how well the model fits the data it is learning from

Validation loss:

- tells us how well the model generalises to unseen text

Typical interpretations:

- both losses decrease: learning is progressing
- training loss decreases but validation loss stays high: overfitting
- both losses stay high: model is not learning enough

Key idea: training measures fitting, validation measures generalisation.

Step 3: Create the Optimizer

```
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=cfg.learning_rate
)
```

- the optimizer is responsible for updating the model parameters
- `model.parameters()` provides all trainable weights
- `AdamW` is a standard optimizer for Transformer models
- `lr` controls the step size of learning

Interpretation: the optimizer tells PyTorch how to transform gradients into parameter updates.

Step 4: The Main Training Loop

```
start = time.time()
for iter in range(cfg.max_iters):
    if iter % cfg.eval_interval == 0 or iter == cfg.max_iters - 1:
        losses = estimate_loss()
        print(f"step {iter:4d} | train loss {losses['train']:.4f} "
              f"| val loss {losses['val']:.4f}")

    xb, yb = get_batch("train")
    logits, loss = model(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()
```

- periodically evaluate train/validation loss
- get one mini-batch from the training split
- run the model to obtain `logits` and `loss`
- compute gradients with backpropagation
- update the parameters with the optimizer

How `forward()` Fits Inside Training

Inside the training loop, the central line is:

```
logits, loss = model(xb, yb)
```

This calls the model's `forward()` method.

So the training pipeline is:

`xb, yb` → `forward()` → `logits, loss` →

`loss.backward()` → `optimizer.step()`

Key idea:

- the forward pass computes predictions and loss
- the backward pass computes gradients
- the optimizer updates the model parameters

What Do `zero_grad()`, `backward()`, and `step()` Do?

After the loss has been computed:

```
logits, loss = model(xb, yb)
```

the following three instructions drive learning:

- `optimizer.zero_grad(set_to_none=True)` clears old gradients from the previous iteration
- `loss.backward()` computes the gradient of the loss with respect to each parameter
- `optimizer.step()` updates the parameters using those gradients

Interpretation: this is the standard PyTorch training cycle.

Using the Trained Model to Generate Text

```
start_char = " " if " " in stoi else chars[0]
context = torch.tensor([[stoi[start_char]]],
                       dtype=torch.long,
                       device=device)

generated = model.generate(
    context,
    max_new_tokens=cfg.generate_tokens,
    temperature=0.9
)

print(decode(generated[0].tolist()))
```

What happens here?

- choose an initial token
- turn it into a tensor
- call the generation method
- decode token IDs back into readable text

What Determines the Quality of Generated Text?

Generated text depends mainly on:

- **training quality:** better trained models generate more coherent text
- **context:** a richer seed may guide generation better
- **temperature:** controls randomness
- **model size:** larger models can represent more patterns

Temperature intuition:

- low temperature: safer, more repetitive
- high temperature: more diverse, more chaotic

Key idea: generation quality reflects both what the model learned and how we sample from it.

Where Does `get_attention_weights()` Fit?

The method `get_attention_weights()` is also **not** part of training.

It belongs to the **inspection pipeline**:

`input sequence` → `get_attention_weights()` →

attention matrices → visualisation / interpretation

Purpose:

- inspect what each attention head is focusing on
- visualise attention patterns
- help us analyse and explain the model

So this method is an **analysis tool**, not a training component.

Complete Workflow of the Mini-Project

text corpus $\rightarrow$ encode $\rightarrow$ token IDs $\rightarrow$ batches

$\rightarrow$ CharTransformer $\rightarrow$ logits $\rightarrow$ loss $\rightarrow$ parameter updates

$\rightarrow$ trained model $\rightarrow$ generation and analysis

Key idea: the project is one end-to-end learning pipeline.

Recommended Implementation Order

A practical order for coding the mini-project is:

1. vocabulary extraction
2. `stoi` / `itos`, `encode()`, `decode()`
3. tensor conversion and dataset split
4. `get_batch()`
5. token and positional embeddings
6. one attention head
7. multi-head attention
8. feedforward network
9. Transformer block
10. `CharTransformer.forward()`
11. training loop
12. `generate()` and attention inspection

Key idea: build the project from data pipeline to full model.

Minimum Viable Success Criteria

Your implementation is already working reasonably well if:

- the code runs end-to-end without crashing
- tensor shapes are consistent
- training loss decreases
- validation loss is not completely unstable
- generated text can be produced
- attention respects the causal mask

Important: A mini-project does not need a perfect model. It needs a correct and coherent implementation.

How Should We Read the Loss Values?

Suppose during training we print:

```
step 0:  train 4.10, val 4.12
step 500:  train 2.35, val 2.48
step 1000:  train 1.90, val 2.70
```

Interpretation:

- from step 0 to 500: both improve
- from step 500 to 1000: training keeps improving
- but validation gets worse

This suggests:

possible overfitting

Usefulness: these values help decide whether to stop training or change hyperparameters.

What Can We Do If Loss Behaviour Is Bad?

If both train and validation loss are high:

- train longer
- increase model capacity
- check batching and masking

If training loss is low but validation loss is high:

- reduce overfitting
- increase dropout
- reduce model size
- stop training earlier

If loss does not decrease at all:

- check learning rate
- verify target shifting in `get_batch()`
- verify tensor shapes and device placement

Key idea: loss values are not only metrics; they are debugging signals.

Optimizer: AdamW

```
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=cfg.learning_rate
)
```

Role of the optimizer:

- updates model parameters using gradients

Why AdamW?

- widely used in Transformer training
- adaptive learning rates
- includes weight decay regularization

Learning rate:

$$\texttt{lr} = \texttt{cfg.learning\_rate}$$

Training Loop

```python
for iter in range(cfg.max_iters):
    if iter % cfg.eval_interval == 0 or iter == cfg.max_iters - 1:
        losses = estimate_loss()
        print(f"step {iter:4d} | train loss {losses['train']:.4f} | "
              f"val loss {losses['val']:.4f}")

    xb, yb = get_batch("train")
    logits, loss = model(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()
```

At each iteration:

1. sample a training batch
2. compute predictions and loss
3. backpropagate gradients
4. update parameters

Training Loop: Step by Step

1. Get a mini-batch

$(x_b, y_b) \leftarrow$ `get_batch("train")`

2. Forward pass

```
logits, loss = model(xb, yb)
```

3. Reset old gradients

```
optimizer.zero_grad()
```

4. Backpropagation

```
loss.backward()
```

5. Parameter update

```
optimizer.step()
```

Key idea:

- this is the standard gradient-based learning cycle

Common Errors and Pitfalls

Typical mistakes:

- **Shape mismatches**
  - wrong dimensions in attention or batching
- **Device mismatch**
  - tensors on CPU vs model on GPU
- **Mask errors**
  - allowing attention to future tokens
- **Batch indexing bugs**
  - incorrect slicing in `get_batch()`
- **Wrong reshaping for loss**
  - forgetting to flatten logits and targets

Key advice:

Always check tensor shapes at every step.

Measuring Training Time

```
start = time.time()
...
elapsed = time.time() - start
print(f"Training completed in {elapsed:.1f} seconds.")
```

Purpose:

- measure total execution time of training

Why useful?

- compare hardware
- compare model sizes
- compare different hyperparameter choices

Example: Effect of Temperature on Generation

Suppose the seed is:

```
"To be"
```

Possible outputs:

Low temperature:

```
"To be or not to be..."
```

Medium temperature:

```
"To be the king that shall..."
```

High temperature:

```
"To be xq!r the- moon..."
```

Interpretation:

- lower temperature gives more conservative choices
- higher temperature gives more surprising choices

Generation Reuses the Same Model Repeatedly

Generation as repeated application of `forward()`
During generation, the model is not changed.
Instead, we repeatedly do:

current context $\rightarrow$ `forward()` $\rightarrow$ next-token probabilities $\rightarrow$

sampling $\rightarrow$ append token

So generation is:

autoregressive inference

and not training.

Key distinction:

- training updates parameters
- generation only uses the learned parameters

Inspecting Attention Weights

```
sample_text = "The cat is sleeping"
sample_idx = torch.tensor(
    [encode(sample_text[:min(len(sample_text), cfg.block_size)])],
    dtype=torch.long,
    device=device
)

weights = model.get_attention_weights(sample_idx, block_number=0)
print("Attention weights shape:", weights.shape)
```

Goal:

- inspect how the model distributes attention inside one block

Returned shape:

$$(\text{num\_heads}, B, T, T)$$

Verifying the Causal Mask

```
first_head = weights[0, 0].detach().cpu()
print(first_head)

print("\nUpper-triangular part (future attention) should be zero:")
print(torch.triu(first_head, diagonal=1))
```

What is being checked?

- the upper-triangular part corresponds to future positions
- it should be zero after masking

Interpretation:

- the model attends only to past and present tokens

A Good Debugging Strategy

When the model does not work, debug in this order:

1. check `encode()` and `decode()`
2. check shapes of `x`, `y`, embeddings, and logits
3. verify `get_batch()` shifting
4. verify the causal mask
5. verify flattening before cross-entropy
6. check CPU/GPU device consistency

Advice: Print shapes often. Most Transformer bugs are shape bugs.

How Should the Notebook Be Organised?

A clean way to organise the mini-project notebook is:

1. Data utilities

- load the text corpus
- build `chars`, `stoi`, `itos`
- define `encode()` and `decode()`
- convert text into a tensor
- split into training and validation sets
- define `get_batch()`

2. Model components

- define `Config`
- implement `Head`
- implement `MultiHeadAttention`
- implement `FeedForward`
- implement `Block`
- implement `CharTransformer`

How Should the Notebook Be Organised?-II

3. Training script

- create the model
- create the optimizer
- define `estimate_loss()`
- run the training loop
- store or plot loss values

4. Generation script

- choose a seed token or seed text
- call `model.generate(...)`
- decode generated token IDs back to text
- optionally inspect attention weights

Key idea: Build the notebook in the same order as the Transformer pipeline:

$$\text{data} \rightarrow \text{model} \rightarrow \text{training} \rightarrow \text{generation}$$

What Should the Final Report Include?

The report should clearly contain:

- dataset used
- main hyperparameters
- training-loss curve
- at least three generated text samples
- discussion of temperature
- discussion of at least two hyperparameters
- short analysis of model behaviour

Goal: show not only that the code runs, but also that you understand what it is doing.

Final Takeaway

In this mini-project you combine:

- tokenisation
- embeddings
- masked self-attention
- multi-head attention
- Transformer blocks
- cross-entropy training
- autoregressive generation

Big picture: A character-level Transformer learns to predict the next token by turning sequences into vector representations, processing them with attention, and improving its parameters through gradient-based learning.

That is the full logic behind modern autoregressive language models.